AI OS — Team Guide

The method the founder uses to build with AI, written down — how one person's disciplined AI-assisted development loop becomes the team's default.

Track: Builders (terminal + coding agent) **Companion:** the Operator Guide for non-terminal roles

Status: internal — not for public surfaces

This is not a tool. It's a *loop* — six practices that turn an AI assistant and a coding agent from a clever autocomplete into a reliable production partner. Followed loosely, they save a little typing. Followed as a system, they remove the two things that quietly wreck AI-assisted work: re-explaining your context every session, and letting your record of what's true drift out from under you.

Most of the value is in the first four practices, and most people skip them because they feel like overhead before the "real" work. They aren't overhead. They're what makes the real work repeatable.

And if the word **Notion** just made you wince — you're a terminal person, not a docs-app person — read §2 first. You will almost never open Notion by hand. It's a structured store your harness reads and writes over MCP, under a protocol you hand the agent once. The discipline is instructions in a file, not clicks in an app.

§0 The two tools — start here

Before any of the practices below, get the ground floor right. This whole method runs on **two tools**; everything else is how you make them work together. If unfamiliar tooling makes you uneasy, this is the one section to

absorb before starting — the rest becomes obvious once these two are in place.

1 · The desktop assistant

`planner + verifier · the cockpit`

- Where you **plan, write specs, and verify** — in plain language. Claude on desktop, or any capable chat app.
- **Install:** download the desktop app and sign in. That's it.
- **How the method loads:** you make one scoped Project per surface and pin its instruction doc (Practice 1). That instruction *is* how it knows your rules.

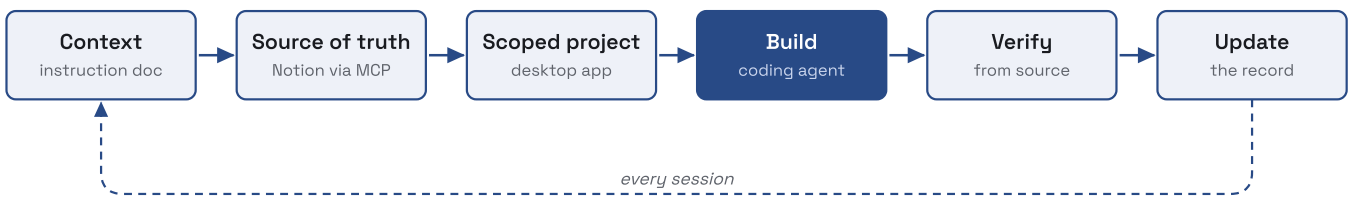
2 · The coding agent

`builder · runs in the terminal`

- Where code actually gets **written**. Team preference order: **opencode** (default) → **Antigravity** → **Claude Code** → an OpenAI coding agent (rarely).
- **Install:** install the CLI/IDE for the one you pick; per-harness file names and launch command are in the §3 table.
- **How the method loads:** it reads `AGENTS.md` first (Claude Code: symlink `CLAUDE.md → AGENTS.md`), and the AI OS team skills — `dev-orchestrator` and the rest — sit in the harness's skill directory so the agent can trigger them.

Wire two MCP servers — that's the plumbing

Notion is the record (§2): the agent reads and writes your project state and canonical facts over it. **Filesystem** lets the desktop assistant read your repo directly, so it can verify the agent's work *from source* instead of trusting a summary. Wire both once per harness — exact config is in §3. With those two connected, the loop below is the entire method.



Practices 1–3 set up and maintain the record. Practices 4–5 do the work. Practice 6 (a desktop-to-agent bridge) is optional and comes last.

§1 The six practices

1 Front-load the context

Essential

Before any work on a surface — a product, an automation, a repo — write a **project-instruction document** that states scope, constraints, conventions, and the references that never change. It gets pinned to the project and read at the start of every session.

Why it matters

The model can't infer your team's rules. If they aren't written down, you re-brief from zero every session and the output drifts toward generic. This document is the single highest-leverage artifact in the method: a weak one makes every later practice weaker.

Do this

Use the template in §4. One document per surface. Keep it scoped — what's in, what's out, the constraints that must never break, the IDs that never change. Revise it when a rule changes, not mid-task.

2 Keep one source of truth

Essential

One canonical place — Notion — holds each project's live state and durable facts. Your harness reads it before asserting anything about where a project stands, and never lets a chat's memory or a stale copy override it. Every line is timestamped; the **most recent timestamp wins**.

Why it matters

Work spreads across many chats and many days. Without one authority, two sessions hold contradictory state and nobody can tell which is real. The record is the arbiter — not your memory, and not the assistant's.

The part that matters for you

You will rarely open Notion by hand. It's a structured store your harness reads and writes over MCP, following a protocol you hand the agent once. §2 gives the exact schema and the wiring.

3 Update the record at session end — only when it moved

Essential

At the end of a working session, the state entry is rewritten **if and only if** Focus, Decisions, Next action, or Blocked-on actually changed. Durable decisions get appended to a log. If nothing moved, the entry and its timestamp are left alone.

Why it matters

The record reports where attention has and hasn't gone. Bumping a timestamp on a session that changed nothing turns that signal into noise. A stale-but-honest date is information — it tells the next person where the work hasn't been.

Do this

Run the four-line check in §6. Decide what's *canonical* with the promotion test: a fact earns a permanent place only if it's still true in ~90 days **and** at least one other person or project would act differently knowing it. Otherwise it's a log line, or nothing. Nothing is usually right.

4 Open a dedicated, scoped project in the desktop app

Essential

Create a **Project** in the desktop assistant (Claude Projects; the ChatGPT equivalent is a Custom GPT — the pattern ports). Paste the instruction doc from Practice 1 as the project's instruction. Attach the reference files it needs. One project per surface.

Why it matters

A scoped project with a pinned instruction makes the assistant behave consistently without re-briefing. It's where specs get authored and work gets managed — the cockpit, not the workshop.

Do this

Follow §3. Keep scope tight; resist the mega-project. Attach only reference input that's actually load-bearing — every extra file dilutes attention.

5 Desktop plans and verifies; the coding agent builds

Core — mechanism varies

The desktop project is the **planner and verifier**. The coding agent — **opencode, Antigravity, or Claude Code** (§3) — is the **builder**. You author the prompt in desktop, the agent executes it, you verify the result **from source** in desktop, and a human pushes after verification. The agent never pushes.

Why it matters

This keeps a verification gate between generation and your repository. The builder can be confidently wrong; the gate is where you catch it. Verifying from the agent's own summary defeats the gate — read the actual files.

Do this

Plan in desktop → paste to the agent → agent builds → verify from source in desktop → capture a revert target → human commits and pushes. Three rules don't bend: **the agent never pushes**; **you verify from source, not from a summary**; **you capture a revert target before pushing**. The full walk is the Builder Playbook (companion doc).

6 Bridge the desktop app to the agent

Advanced — optional, do last

A script that connects the desktop app to the coding agent so hand-off is automated (on macOS, an `osascript`/AppleScript bridge).

Why you can skip it

It's the most fragile and platform-specific piece, and the least necessary. Use the coding agent directly, connect tools with MCP, and paste by hand. A bridge that drives the GUI by injecting keystrokes breaks the moment a window moves. Add one only after the loop is reflex, and prefer a CLI or MCP path over screen-driving. Cross-platform recipes are in Appendix A.

§2 What context lives where

The one thing that trips everyone up: there are **two kinds of context**, and they live in two different places. Mix them and you get a mess — company facts buried in a repo nobody else can see, or a single project's build spec clogging the shared record. Keep them separate.

Standing context → Notion

`durable · cross-project · read by any session`

- The **company**: what Softograph is, brand and identity rules, the do-not-use list, conventions that apply to all work.
- Your **work across every repo**: the current state of each project (one row each), the decisions you've made and why, the IDs that never change.
- Anything you'd still need if a repo vanished tomorrow.

Project context → the repo

`specific · per-project · read by the coding agent`

- What this one product **is and does**: BRD / PRD.
- How it's **shaped**: SCHEMA, and the specs (architecture, API).
- The **build plan and progress**: the playbook features, PROGRESS.md, CLAUDE.md.
- Things that only make sense inside this repo, and die with it.

The filing rule — say this to a confused dev

Would you still need it if this repo vanished? **Yes → Notion.** Is it "what this one product is and how to build it"? **Repo.** The company, your project states, your decisions, and the canonical IDs live in Notion. The BRD, the schema, and the specs live in the repo. That's the whole distinction.

The three Notion stores

Standing context is held in three stores, all driven by your agent over the **Notion MCP server**. You set them up once and tell the agent how to use them; after that it reads at session start and writes at session end without you clicking anything. IDs below are placeholders — use your own.

1 · Project State

`Notion database · one row per project · read at start, rewritten at end (only if it moved)`

The live snapshot. This is what any session — or any teammate — reads to know exactly where a project stands right now.

```
Focus — one line: what's in hand Decisions (last 3) — recent calls, newest first
Next action — the single next move Blocked on — one line, or "—" Updated — date
(do not bump if nothing changed) Pointers — issue IDs · workflow IDs · repo paths
```

2 · Decision Log

`Notion database · append-only · never edited in place`

The history. Every durable decision lands here with a date and a scope. Newer decisions supersede older ones by pointer, not by deletion — history is useful. This is where execution IDs, commit SHAs, and root-cause notes go — never in Project State.

`Statement` — the decision, one line (title) `Date` — when `Scope` — Local | Cross (does another project care?) `Supersedes` — relation to the row this overrides `Superseded by` — back-filled automatically

3 · Canonical Facts

`Notion page or small database · the authority · guarded`

The things that don't change: IDs, conventions, and the do-not-use list. The agent cross-references this before asserting a fact. Getting something *into* Canonical Facts goes through a review gate — the promotion test — so what's here stays trustworthy.

What to actually write — copy these

The next question is always "fine, but what do I *put* in it?" Here's each store filled with real values from the Acme Portal run. Copy the shape, swap the content. Two rules save you: date everything, and never type the do-not-use list or the canonical IDs from memory — pull them from the live page.

Project State — one row, filled:

```
Focus         Auth flow: email+password login with session tokens
Decisions     - [2026-09-16] JWT over server sessions — stateless, simpler deploy
              - [2026-09-15] Postgres over SQLite — need concurrent writes
              - [2026-09-14] Next.js app-router, not pages
Next action   Wire /login to the token endpoint; add refresh rotation
Blocked on    —
Updated       2026-09-16
Pointers      repo: acme-portal · issue: ACME-12 · feature: FEATURE_01_auth
# Focus and Next action are one line each. Keep only ~3 recent decisions here —
# older ones live in the Decision Log, not clogging the state.
```

Decision Log — append-only, two entries:

```
Statement    JWT over server sessions for Acme Portal auth
Date         2026-09-16
Scope        Local          # only this project cares
Supersedes   —

Statement    Automations write to the Notion review queue, never the task spine
Date         2026-09-11
Scope        Cross          # every automation project follows this
Supersedes   —
# Scope = Cross is also the bar for logging at all: would another project act
# differently knowing this? If no, it's probably not worth a row.
```

Canonical Facts — the authority, an excerpt:

```
## Conventions (apply to all work)
- Automation code nodes: ES5 only — var, string concat, explicit for-loops.
- Email from automations: the shared Gmail node, never SMTP.
- People referred to by title in any output, never by name.

## Canonical IDs   — copy real values from the live page, never from memory
- n8n host:           <url>
- Prospect DB:        <id>
- RFP tracker sheet:  <id>

## Do-not-use (permanent)   — lives on the live shared page; it changes, memory c
- <pull the eliminated channels/tools list from the page — don't retype it here>
```

Blank template — start from this:

```
# Project State — one row per project
Focus         <one line: what's in hand right now>
Decisions     - [<date>] <recent call + one-clause why>
Next action   <the single next move>
Blocked on    <one line, or —>
Updated       <date — do NOT bump if nothing changed>
Pointers      repo: <name> · issue: <id> · feature: <file>

# Decision Log — append a row when you decide something durable
Statement     <the decision, one line>
Date          <date>
Scope         Local | Cross
Supersedes    <old row, or —>
```

Wiring it to your harness

Two portable conventions make this work on any coding agent — opencode, Antigravity, or Claude Code (full table in §3):

1 · The protocol goes in `AGENTS.md`. That's the instruction file opencode and Antigravity read natively; for Claude Code, keep the same file and symlink `CLAUDE.md → AGENTS.md`. One source, every tool. The record protocol lives there as plain rules the agent follows — not clicks you make:

```
# AGENTS.md — the record protocol
SESSION START  read Project State <db-id> and Canonical Facts <page-id>.
DURING         cross-reference Canonical Facts before asserting a fact.
               on conflict, the most recent timestamp wins.
SESSION END    rewrite Project State only if it moved; append durable
               decisions to Decision Log <db-id> with Scope = Local | Cross.
```

2 · Connect the Notion MCP server to whatever's doing the work. Same server, but the config lives in a different place per harness:

```
# Claude Code — .mcp.json (project root)
{ "mcpServers": { "notion": { "type":"http", "url":"https://mcp.notion.com/mcp" }

# opencode — the "mcp" section of opencode.json  (see opencode docs for exact key
# Antigravity — ~/.gemini/config/mcp_config.json  — note: "serverUrl", NOT "url"
{ "mcpServers": { "notion": { "serverUrl":"https://mcp.notion.com/mcp" } } }
```

That's the whole trick: the structure is a database, the discipline is `AGENTS.md`, and the agent does the reading and writing over MCP. **You review; you don't transcribe.** For *why* this records reliably with no scheduler, see the **Reference System** doc, §3.

> **First five minutes, on a new project**
>
> The full walk is the Builder Playbook. The shape (coding agent shown as opencode — swap per the §3 table):
>
> ```
> # 1 · readiness — ask the agent to run the dev-orchestrator pre-flight chec
> #     (references/claude-code-setup-guide.md, Part 8): Node, Git, SSH, agen
>
> # 2 · scaffold — trigger the dev-orchestrator skill; it reads
> #     references/project-archetypes.md and lays down Brain→Soul→Body, the
> #     instruction file, specs/, playbook/ directly. No script to run.
>
> # 3 · hand off to your coding agent, which reads AGENTS.md first
> cd acme-portal && opencode        # or: antigravity · claude
> ```

The `dev-orchestrator` skill does the scaffolding — it reads `references/project-archetypes.md` and creates the tree directly, emitting `CLAUDE.md` as the handoff file; on opencode or Antigravity, symlink or rename it to `AGENTS.md` — same content. From there it's the loop.

§3 Setup — run once

One-time environment setup, in order. Per-project initialisation lives in the Playbook — this is the machine underneath it.

Pick your harness first. The method is identical on all of them; only the file names and the launch command change. This is why the docs are written tool-neutral — substitute your column.

	opencode	Antigravity	Claude Code
Instruction file	`AGENTS.md`	`AGENTS.md`	`CLAUDE.md` (symlink → AGENTS.md)
MCP config	`opencode.json` (mcp section)	`~/.gemini/config/mcp_config.json` — uses `serverUrl`	`.mcp.json`
Launch	`opencode`	Antigravity IDE / CLI	`claude`
Test / ship	the project's test cmd + git	Planning mode → run + git	`/quick-test` · `/commit-push-pr`

Keep one instruction file — `AGENTS.md`. opencode and Antigravity read it natively; for Claude Code, `ln -s AGENTS.md CLAUDE.md`. Everywhere these docs show `CLAUDE.md`, `.mcp.json`, or `claude`, read it as your harness's column. (The method is harness-agnostic by design — the runtime it graduates into drives opencode, Claude Code, headless, and n8n alike.)

1. **Install the desktop assistant and sign in.** This is your cockpit — planning, spec-writing, and verification all happen here. Any capable chat app works.

2. **Install your coding agent** (opencode / Antigravity / Claude Code) and ask it to run the `dev-orchestrator` pre-flight check (`references/claude-code-setup-guide.md`, Part 8) to confirm Node, Git, SSH, and the agent are ready.

3. **Set your agent config** once: a permission preset so you're not approving every command, and — where your harness supports them — the shortcuts you'll reuse (test, commit-and-push, end-session). On Claude Code these are `~/.claude/` slash commands; opencode and Antigravity have their own.

4 **Wire your MCP servers.** Add the two this method needs — **Notion** (the record) and **Filesystem** (lets the desktop assistant read your repo to verify from source) — plus any tools a surface touches, in your harness's MCP config from the table above. Notion lets the agent drive the record; Filesystem is what makes verify-from-source work.

5 **Create the three Notion stores** — Project State, Decision Log, Canonical Facts (§2). One set covers all your projects; Project State just gets a new row each time.

6 **Create a scoped desktop Project** per surface and paste its instruction doc (§4 template). Attach only load-bearing references.

7 **Prove the loop on something small** before trusting it — context → project → build → verify from source → update the record. Fix anything unclear now, on a task that doesn't matter.

§4 Project-instruction template

Copy this into a new project. Fill every `<placeholder>`. Delete sections that genuinely don't apply — but think twice before deleting Constraints or Canonical references; those are what stop drift.

```
# Project Instruction — <surface name>

PURPOSE
<One or two sentences: what this surface is and what it's for.>

SCOPE
In:  <what this project covers>
Out: <what it explicitly does NOT cover — route elsewhere>

HARD CONSTRAINTS  # never break, even on request
- <cost rule, e.g. free tier / open-source only>
- <stack rule, e.g. AI synthesis = <model> only>
- <channel rule: what's permanently off the table>

TOOL & TECH CONVENTIONS
- <language/style rules the agent must follow>
- <integration/node conventions>
- <error-handling / naming conventions>

THE RECORD  # the three Notion stores, over MCP
- Project State: <db-id>   Canonical Facts: <page-id>
- Decision Log:  <db-id>
- Protocol: read at start · cross-reference before asserting · write at end if mc

CANONICAL REFERENCES
- Repos: <paths — mark which are source vs build output>
- Task spine: <where issues live>

WORKING STYLE
- Execute, don't interview. State assumptions after acting.
- Report state, not narrative. Verify from source, not summaries.
- The coding agent never pushes; a human pushes after verification.
```

§5 Worked example

A filled instruction for a plausible surface — a marketing-content scanner automation. IDs are placeholders on purpose; never hardcode live IDs into a template others copy.

```
# Project Instruction — Marketing Content Scanner

PURPOSE
An automation that scans configured sources on a schedule, scores items for
relevance, and files the keepers for a human to review. No auto-publishing.

SCOPE
In:  the scanner workflow, its scoring step, and the review queue it writes to.
Out: anything that sends outbound messages or publishes — separate surfaces.

HARD CONSTRAINTS
- Zero paid services. Free tiers and open-source only.
- AI synthesis = one designated model only. Not a mix.
- Automations never write to the task spine. They write to the review queue.

TOOL & TECH CONVENTIONS
- Code nodes: the team's baseline dialect only — no newer syntax.
- IF/Filter nodes: strict type validation + explicit combinator, always.
- Workflow name: "<Team> Content Scanner — <frequency>". No issue-id prefix.

THE RECORD
- Project State: <db-id>   Canonical Facts: <page-id>   Decision Log: <db-id>

WORKING STYLE
- People are referred to by title, never by name, in any output.
- Execute on reasonable assumptions; state them at the end. One question max.
```

§6 The session-end check

Four questions, thirty seconds. This is what keeps the record trustworthy.

- ☐ **Did state actually change?** If Focus, Decisions, Next action, or Blocked-on moved, rewrite the row in full and stamp it today. If not, leave it — and leave the date.
- ☐ **Any durable decision?** Append it to the Decision Log with today's date. If it overrides an older one, set Supersedes rather than deleting.
- ☐ **Is anything canonical?** Run the promotion test — true in ~90 days *and* changes another actor's behavior. Both, or it's not canonical.

- [ ] **Did this session write files into a source repo?** If yes, commit and push. If it was planning only, don't. Obligations attach to artifacts, not to conversations.

§7 Graduating to the runtime

Once the manual loop is reliable, some tasks become clean enough to hand off whole. That's when you write a **spec** the runtime can run instead of steering the build by hand.

A spec declares a goal, at least one mandatory **verifier**, its constraints, and how much autonomy it's allowed. The runtime decomposes it into a plan, binds each step to a real capability (an unmet need surfaces as a gap — never a made-up binding), runs it, and checks the result against the verifier. The **verifier's verdict outranks the agent's self-report**, and autonomy is *earned* on clean-run evidence: a held task holds, an escalation demotes. Same paradigm at both scopes — the full set for you, a team-scoped slice for the team.

Don't start here. Start with the loop. The runtime is what the loop becomes once you trust it — and a good spec is only as good as the scoped context (Practice 1) it's written in.

A Appendix — the bridge, cross-platform

Only if you've decided you want it (Practice 6). The principle on every platform: **prefer a CLI or MCP call over driving the GUI.** Keystroke injection into a chat window is a last resort, not a design.

macOS

`osascript` (AppleScript / JXA) can activate the app, insert text, and manage windows — the most mature option because the accessibility APIs are stable. Still reach for the agent's CLI first.

Linux

Under X11, `xdotool` handles keystrokes and windows; under Wayland, `ydotool` or `wtype` for input and `wmctrl` for windows. Wayland

deliberately restricts synthetic input, so a CLI/MCP path is even more clearly right here.

Windows

`AutoHotkey` is the pragmatic choice; PowerShell with `System.Windows.Automation` is the more robust one. Or sidestep GUI automation entirely by running the agent CLI-first under WSL.

> **Honest assessment**
>
> This bridge is the piece most likely to be unnecessary. MCP connects tools directly, the coding agent is a first-class CLI, and the runtime handles declared specs. If you're building elaborate window automation, step back and ask whether an MCP server or a direct CLI call does the same job without the fragility.

B Appendix — for maintainers

> **Where this document lives**
>
> Author and version it in the **master** repository under `docs/` (flat — the team-sync allowlist copies per file). Let the team-sync script carry it into the team distribution. **Never edit it directly in the team repo** — that repo is a build output; direct edits get overwritten on the next sync and break the derivation contract. You do not version a build output.

> **Internal only**
>
> This guide contains operational specifics. It stays internal and off the public hub. A public version — if one is ever wanted — is a separate, sanitized, reasoning-topology-only piece that exposes no wiring, routed through the standing publishing pipeline. Two artifacts, never merged.